

Rigid Body Attitude Control Simulation with Reaction Wheels

Caleb Oblepias

March 26, 2026

1 Overview

This project implements a full end-to-end simulation of spacecraft attitude dynamics and control using quaternion representations and reaction wheel actuation. The system models rigid body physics, applies a closed-loop PID controller, and incorporates realistic actuator constraints including torque limits, speed limits, and saturation masking.

The simulation is structured to serve as a high-fidelity truth model for deterministic real-time implementations such as FPGA-based systems.

2 System Modules

The implementation is composed of the following modules:

- **attitude_error.py**: quaternion operations and attitude error computation
- **controller.py**: PID control law with anti-windup
- **reaction_wheels.py**: torque allocation, wheel dynamics, and body torque generation
- **rotational_dynamics.py**: rigid body dynamics and IMU emulation
- **quaternion_formation.py**: quaternion kinematics and propagation
- **rk4.py**: numerical integration (Runge-Kutta 4th order)
- **main.py**: simulation loop and system integration

3 Mathematical Foundations

3.1 Rigid Body Dynamics

The rotational motion of the spacecraft is governed by Euler's equation:

$$\dot{\omega} = I^{-1} (\tau - \omega \times (I\omega)) \quad (1)$$

3.2 Quaternion Kinematics

$$\dot{q} = \frac{1}{2} \Omega(\omega)q \quad (2)$$

3.3 Control Law

$$\tau = -K_p e - K_d \omega - K_i \int e dt \quad (3)$$

4 End-to-End Control Loop Execution

Each timestep of size dt executes the following pipeline.

4.1 1. Attitude Error (`attitude_error.py`)

The error between desired and current attitude is computed.

4.1.1 Quaternion Conjugate

$$q^{-1} = [q_0, -q_1, -q_2, -q_3] \quad (4)$$

4.1.2 Quaternion Multiplication

$$q_{\text{err}} = q_{\text{des}} \otimes q^{-1} \quad (5)$$

4.1.3 Error Vector Extraction

$$e = q_{\text{err}}[1 : 3] \quad (6)$$

This vector represents the axis-angle error approximation used for control.

4.2 2. PID Controller (`controller.py`)

The controller computes the required corrective torque.

4.2.1 Integral Update

$$e_{\text{int}} \leftarrow e_{\text{int}} + e \cdot dt \quad (7)$$

4.2.2 Anti-Windup Clamping

$$e_{\text{int}} = \text{clip}(e_{\text{int}}, -e_{\text{max}}, e_{\text{max}}) \quad (8)$$

4.2.3 Control Torque

$$\tau_{\text{cmd}} = -K_p e - K_d \omega - K_i e_{\text{int}} \quad (9)$$

4.3 3. Torque Allocation (`reaction_wheels.py`)

The commanded torque is distributed across reaction wheels.

$$\tau_w = A^\dagger \tau_{\text{cmd}} \quad (10)$$

A negative sign is applied in implementation to reflect equal and opposite torque on the spacecraft.

4.4 4. Actuator Constraints and Saturation Handling

4.4.1 Torque Clipping

$$\tau_w = \text{clip}(\tau_w, -\tau_{\max}, \tau_{\max}) \quad (11)$$

4.4.2 Directional Saturation Mask

If a wheel is already at maximum speed, it cannot accelerate further in that direction.

$$\text{mask}_i = \begin{cases} 1 & \text{if } \omega_i \geq \omega_{\max} \text{ and } \tau_i > 0 \\ 1 & \text{if } \omega_i \leq -\omega_{\max} \text{ and } \tau_i < 0 \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

$$\tau_w[i] = 0 \quad \text{if } \text{mask}_i = 1 \quad (13)$$

4.5 5. Reaction Wheel Dynamics

4.5.1 Wheel Acceleration

$$\dot{\omega}_w = \frac{\tau_w}{J_w} \quad (14)$$

4.5.2 Euler Integration

$$\omega_w^{next} = \omega_w + \dot{\omega}_w dt \quad (15)$$

4.5.3 Speed Clipping

$$\omega_w^{next} = \text{clip}(\omega_w^{next}, -\omega_{\max}, \omega_{\max}) \quad (16)$$

4.6 6. Body Torque Computation

The spacecraft experiences torque equal to the negative change in wheel angular momentum:

$$\tau_{\text{body}} = -A \left(J_w \frac{\omega_w^{next} - \omega_w}{dt} \right) \quad (17)$$

4.7 7. Rigid Body Propagation (rotational_dynamics.py)

The system uses RK4 integration for angular velocity.

4.7.1 Dynamics Function

$$\dot{\omega} = I^{-1} (\tau - \omega \times (I\omega)) \quad (18)$$

4.7.2 RK4 Integration

$$k_1 = f(\omega) \quad (19)$$

$$k_2 = f(\omega + \frac{1}{2}dt k_1) \quad (20)$$

$$k_3 = f(\omega + \frac{1}{2}dt k_2) \quad (21)$$

$$k_4 = f(\omega + dt k_3) \quad (22)$$

$$\omega_{next} = \omega + \frac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (23)$$

4.8 8. Quaternion Propagation (quaternion_formation.py)

4.8.1 Quaternion Dynamics

$$\dot{q} = \frac{1}{2}\Omega(\omega)q \quad (24)$$

4.8.2 RK4 Integration

The quaternion is propagated using the same RK4 method.

4.8.3 Normalization

$$q \leftarrow \frac{q}{\|q\|} \quad (25)$$

This prevents numerical drift.

4.9 9. State Update

The following states are updated:

- Angular velocity ω
- Quaternion q
- Reaction wheel speeds ω_w

4.10 10. Attitude Error Metric

The scalar rotation error is computed as:

$$\theta = 2 \cos^{-1}(|q_{des} \cdot q|) \quad (26)$$

4.11 11. Logging and Visualization

The simulation logs:

- Angular velocity
- Quaternion state
- Wheel speeds

- Attitude error magnitude

These are plotted over time to evaluate system behavior.

5 Numerical Integration Module (rk4.py)

The RK4 integrator is defined as:

$$k_1 = f(x) \tag{27}$$

$$k_2 = f(x + \frac{1}{2}dtk_1) \tag{28}$$

$$k_3 = f(x + \frac{1}{2}dtk_2) \tag{29}$$

$$k_4 = f(x + dtk_3) \tag{30}$$

$$x_{next} = x + \frac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{31}$$

This integrator is reused across both rotational dynamics and quaternion propagation.

6 Design Considerations

- RK4 improves numerical accuracy over Euler integration
- Pseudoinverse enables overactuated control
- Saturation masking enforces physically realizable actuator behavior

7 Next Steps

- FPGA implementation of control pipeline
- Fixed-point arithmetic conversion
- Sensor noise modeling
- State estimation (EKF)
- Fault tolerance for actuator failures

8 Conclusion

This simulation provides a complete and physically consistent implementation of spacecraft attitude control. The structure and modularity of the system make it well-suited for extension into real-time embedded and hardware-accelerated platforms.